CSC 251 Lecture 05: Sequences and Slicing

Sequences

A sequence is a built-in object type that holds multiple items of data. Six types of sequence objects in Python:

- mutable – lists, bytearrays
- immutable – tuples, strings, byte sequences, range objects

Range objects

The range function is a constructor that produces an immutable sequence of numbers using 1, 2, or 3 arguments, a **start, stop,** and **step**. These iterable values are generated lazily (on-demand).

Example 1a: Recall how we iterated with a for loop and output the range object sequence:

```
for f in range(1, 10, 2):
    print(f, end = " ")
```

```
1 3 5 7 9
```

Example 1b: A range object can be assigned to a variable or can be output, which returns the function call to create the object.

doesn't generate sequence items until needed

```
print(range(1, 10, 2))
```

```
range(1, 10, 2)
```

Example 1c: It must be converted to a list or tuple to see the values or it can be unpacked with *:

```
my_range = range(1, 10, 2)
print(*my_range)
```

needed when unpacked

```
1 3 5 7 9
```

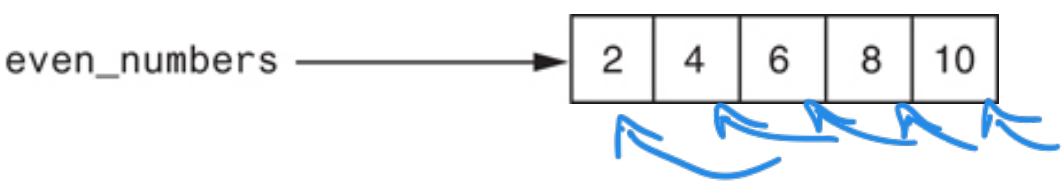
Lists

A list is an ordered, mutable sequence object with multiple data *elements* that can be changed during execution.

Example 2a: Elements are enclosed in square brackets, separated by commas.

```
even_numbers = [2, 4, 6, 8, 10]
```

where the variable `even_numbers` will reference the list in memory.



reference to each int object held

Example 2b: Python allows different data types to be held in the same list:

```
student_info = ['Leonard', 25, 65.43]
```

heterogeneity allowed can even include elements that are lists

What's happening in memory? (`listSizeDemo.ipynb`)

lots of overhead for list itself

Example 2c: Python allows for convenient printing of a list:

```
print(student_info)
```

```
['Leonard', 25, 65.43]
```

Tuples

A tuple is an ordered, immutable collection of any type elements.

Example 3a: How does Python know the difference between a list and a tuple?

```
dog_list = ['Westie', 'Collie', 'Beagle']
dog_tuple = ('Westie', 'Collie', 'Beagle')

num_list = [10, 20, 30]
num_tuple = 10, 20, 30
```

tuples defined with or without ( )

Example 3b: How do you distinguish a single element tuple? Why would you want one?

```
my_tuple = (10,)      or    your_tuple = 3.14,
```

← the comma is the key

Strings

Example 4: A string is an ordered, immutable sequence of Unicode characters enclosed in single or double quotes (or triple quotes for multiline strings).

```
game = 'Wingspan'     or    letter = "a"    or    mascot_name = "Ms. Wuf"
```

Constructors

Example 5a: The `list()` function is a constructor that can convert an iterable sequence object into a list:

```
lnums = list(range(6, 0, -2))
print(lnums)                                    [6, 4, 2]
```

Example 5b: Likewise, use `tuple()` to convert to a tuple:

```
tnums = tuple(lnums)
print(tnums)                                    (6, 4, 2)
```

Notebook output w/o print, shows string type

Example 5c: Or `str()` to convert to a string:

```
snums = str(tnums)
print(snums)                                    (6, 4, 2) or '(6, 4, 2)'
```

terminal output

Indexing Lists, Tuples, and Strings

Example 6a: The elements within ordered sequences can be specified by their **index**, starting with an index of 0.

```
fives = [5, 10, 15]
print(fives[0], fives[1], fives[2])      5 10 15

print(dog_tuple[1])                      Collie
```

Example 6b: Ordered sequences can be indexed from the **end** of the data structure using negative indexes:

```
poetry = 'Roses are red'
print(poetry[-1], poetry[-7], poetry[10])     d a r
```

Example 6c: Caution! An invalid index will yield an `IndexError` exception:

```
print(fives[-1], fives[-2], fives[-4])
IndexError: list index out of range
```

Iterating over Sequences

Example 7a: The **target variable** will reference a copy of an element in the named sequence each iteration.

```
for num in fives:
   print(num)
```

5
10
15

```
for ch in game:
   print(ch, sep = '-')
```

w-i-n-g-s-p-a-n

Example 7b: Any problem here? Solution?

```
index = 0
scores = [100, 78, 83, 91]
while index < 5:
   print(scores[index])
   index += 1
```

use len(scores) to avoid index errors as list may grow/shrink

or more Pythonic

```
for score in scores:
   ...
```

Sequence Concatenation

Example 8a: Sequences can be concatenated using the `'+'` or the `'+='` operator:

```
scores1 = [100, 78, 91]
scores2 = [94, 71, 67, 95]
all_scores = scores1 + scores2
print(all_scores)                          [100, 78, 91, 94, 71, 67, 95]

scores3 = [69, 74, 91, 96]
all_scores += scores3
print(all_scores)                [100, 78, 91, 94, 71, 67, 95, 69, 74, 91, 96]
```

Example 8b: Concatenating strings

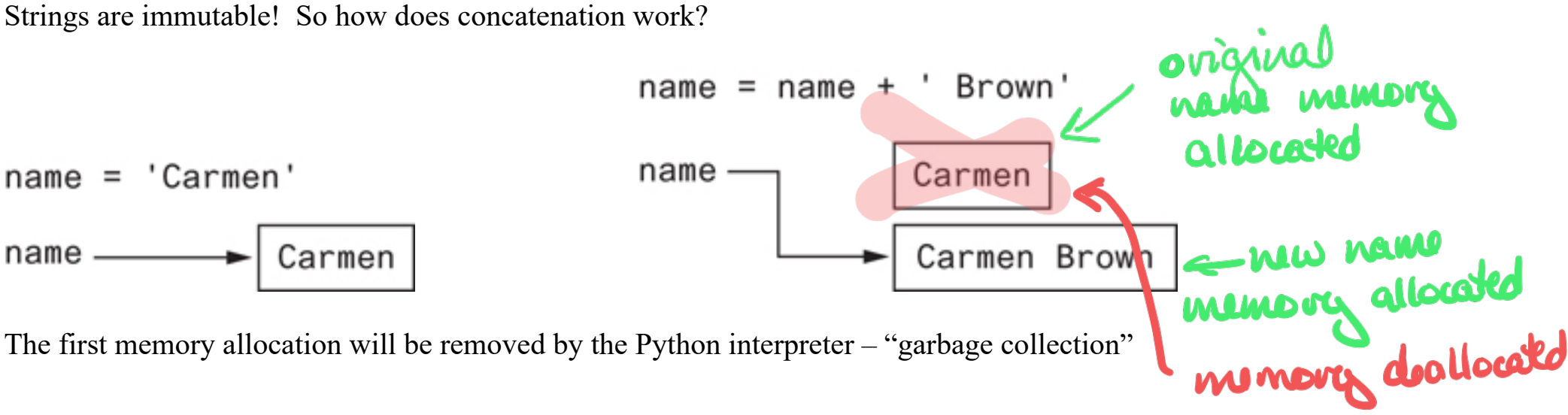
```
first = 'Comp'
second = 'Sci'
first += last
print(first)                               CompSci
```

Better:

```
major = first + ' ' + second
```

```
print(major)                                    Comp Sci
```

Strings are immutable! So how does concatenation work?



The first memory allocation will be removed by the Python interpreter – "garbage collection"

Mutable Sequences

Example 9: List elements can be modifed using assignment statements:

```
scores = [100, 78, 83, 91]
scores[1] = 88
print(scores)                                   [100, 88, 83, 91]
```

Example 10: What about strings or tuples?

```
first = 'think'          or          dog_tuple[0] = 'Poodle'
first[4] = 'g'
```

errors! They are immutable!

Example 11: While you can't change an existing immutable sequence, you can create a new one from others:

```
new_tuple = num_tuple + dog_tuple
```

or just reassign a tuple:

```
num_tuple += dog_tuple
```

work around

Example 12a: You can delete an entire list, tuple, or string from the namespace:

```
del new_tuple
```

Example 12b: But, you can only delete individual elements in mutable sequences:

```
del fives[0]
```

The Repetition Operator

Example 13a: The * symbol is called the **repetition operator** and makes copies of a sequence.

```
values = [0, 1, 2 3]
values3 = values * 3
print(values3)                  [0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]
```

Example 13b: What is output here?

```
copy = ["go"] * 5
print(copy)
```

["go", "go", "go", "go", "go"]

Example 13c: Which do you want?

```
repeat = [10] * 5
```

[10, 10, 10, 10, 10]

versus

```
repeat = [10 * 5]
```

[50]

Example 14: A common practice is create a list of 0 values, then update each:

```
sales = [0] * NUM_DAYS

print('Enter the sales for each day.')
for index in range(len(sales)):
    sales[index] = float(input(f'Day #{index + 1}: '))
```

Unpacking/Packing Sequences

Example 15a: You may unpack the items in a tuple through a multiple assignment statement to improve code r eadability, convenience, and as an alternative to iteration.

```
dog_tuple = ('Westie', 'Collie', 'Beagle')
print(dog_tuple[0], dog_tuple[1], dog_tuple[2])
```

or

```
d1, d2, d3 = dog_tuple
print(d1, d2, d3)
```

← more Pythonic

Example 15b: Recall from Lecture 04 when returning multiple values from functions:

```
def get_name():
    first = input("Enter first name: ")
    last = input("Enter last name: ")
    . . .
    return first, last
```

← a tuple w/o ( )

In main:

```
fname, lname = get_name()
```

← unpacked

Example 15c: You may unpack using a starred variable:

```
vals = [0,1,2,3,4,5,6,7,8,9]
first, *mid, last = vals
print(first, mid, last)
```

0 [1, 2, 3, 4, 5, 6, 7, 8] 9

Example 16: Python has a built-in zip option to combine sequences into an iterator. They can be "unzipped" by converting to a list or tuple. Helpful for iterating over multiple sequences or when building dictionaries.

```
tA = (1, 2, 3)
tB = ('a', 'b', 'c')
tz = zip(tA, tB)
print(tuple(tz))
```

← unzips back into tuples

Searching a List with in and not in

Example 17a: Recall the **Sequential/Linear Search** which checks each element in the list against the search item to determine if it's in the list, using a Boolean flag?

```
numbers = [10, 20, 30, 40, 50, 60]
found = False
search_value = 30

for index in range(len(numbers)):
  if numbers[index] == search_value:
     found = True
. . .
```

Example 17b: More Pythonic (and readable) - use the `in` operator to determine if an item is contained in a list.

```
prod_nums = ['V475', 'F987', 'Q143', 'R688']
search_item = input('Enter a product number: ')

if search_item in prod_nums:
  print(f'{search_item} was found in the list.')
else:
  print(f'{search_item} was not found in the list.')
```

Given the following input:
`F989`
`R688`

Corresponding output:
`F989 was not found in the list.`
`R688 was found in the list.`

The `not` operator can also be used with `in`:

```
if search_item not in prod_nums:
    print(f'{search_item} was not found in the list.')
```

Example 15c: We can use `in` and `not in` with strings as well:

```
state = 'Mississippi'
if 'is' in state:
   print("'is' was found")

if 'at' not in state:
   print("'at' was NOT found")
```

Slicing

Slice – a span of elements selected from a sequence according to the format:

sequence`[`*start*`:`*end*`]`

This syntax returns a sequence containing a copy of the elements from the start index up to **but not including the end index**. Recall how this is consistent with `range(2,5)` which yields `2 3 4` but doesn't include 5.

Given the following list:

```
numbers = [0, 10, 20, 30, 40, 50, 60, 70]
```

Example 16a: Let's slice the list:

```
print(numbers[1:4])                    [10, 20, 30]
```

Example 16b: Leaving the start index blank begins at the left end of the list:

```
print(numbers[:4])                     [0, 10, 20, 30]
```

Example 16c: Leaving the end index blank slices through the right end of the list:

```
print(numbers[4:])                     [40, 50, 60, 70]
```

Example 16d: And if both indexes are blank:

```
print(numbers[:])                      [0, 10, 20, 30, 40, 50, 60, 70]
```

Example 16e: A step can also be included, like with the `range` function:

```
print(numbers[1:4:2])                  [10, 30]
```

Example 16d: Negative indexes may be used, here starting 3 from the right end, through the right end:

```
print(numbers[-3:])                    [50, 60, 70]
```

Example 16d: Negative steps may be used provided the indexes allow you to slice from right to left:

```
print(numbers[4::-2])                  [40, 20, 0]
```

← to left end since negative step

Example 17a: String slicing follows the same rules:

```
uni_name = 'North Carolina State University'

print(uni_name[6:14])                  Carolina
```

Example 17b:

```
print(yourstring[-6::-1])              evinU etatS aniloraC htroN
```